

Laboratorio 6 (Implementación de hilos).

Oscar Grande, Didier Posse.

Noviembre 2025

1 Ejercicio 1 (Análisis de sentimientos mediante comentarios).

como primer punto realizaremos un ejercicio de análisis de sentimientos, el objetivo es procesar textos en paralelo para analizar emociones usando hilos. Fortaleciendo un código el cual pueda procesar un lote de comentarios y este pueda interpretar a través de hilos los sentimientos positivos o negativos, luego se buscara desarrollar el aplicativo mediante streamlit para su visualización. Una vez hecho esto subirlo a Docker y correr la aplicación.

1.1 Código mejorado y explicación.

- Este código también lo puede encontrar en el repositorio.

```
1 from concurrent.futures import ThreadPoolExecutor
2 from threading import Lock
3 import string
4
5 # Diccionario de palabras positivas y negativas
6 positive_words = ('bueno', 'excelente', 'maravilloso', 'genial', 'fantástico', 'encanto', 'buenísimo', 'lindo', 'bonito')
7 negative_words = ('malo', 'terrible', 'horrible', 'pesimo', 'desastre', 'malísimo', 'horrendo', 'feo')
8
9 # Variable compartida entre hilos
10 results = []
11 lock = Lock()
12
13 def clean_text(text):
14     """Este limpia puntuación para evitar falsos positivos."""
15     return text.lower().translate(str.maketrans("", "", string.punctuation))
16
17 def analyze_comment(comment):
18     """Este analiza un solo comentario y retorna el sentimiento."""
19     cleaned = clean_text(comment)
20     words = cleaned.split()
21
22     sentiment = "neutral"
23     if any(w in positive_words for w in words):
24         sentiment = "positive"
25     elif any(w in negative_words for w in words):
26         sentiment = "negative"
27
28     return (comment, sentiment)
29
30 def analyze_sentiment_chunk(chunk):
31     """Este procesa un subconjunto de comentarios y usa lock para evitar carrera."""
32     local_results = []
33     for comment in chunk:
34         local_results.append(analyze_comment(comment))
35
36     # Guardar resultados en la lista global con protección
37     with lock:
38         results.extend(local_results)
```

Figure 1: Código sentimientos.py

```

40 # Programa como tal
41
42 comments = [
43     "Me encantó este producto",
44     "Fue terrible, no me gustó",
45     "Más o menos, nada especial",
46     "Excelente calidad, lo recomiendo",
47     "Horrible experiencia"
48 ]
49
50 chunk_size = 2
51
52 # Dividir comentarios
53 chunks = [comments[i:i + chunk_size] for i in range(0, len(comments), chunk_size)]
54
55 # Aca lo que hacemos es procesar con hilos usando ThreadPoolExecutor
56 with ThreadPoolExecutor(max_workers=len(chunks)) as executor:
57     executor.map(analyze_sentiment_chunk, chunks)
58
59 print("\nRESULTADOS DEL ANALISIS:")
60 for c, s in results:
61     print(f"> {c} - {s}")

```

Figure 2: Segunda porción de código.

- En el código se utiliza **ThreadPoolExecutor** con el fin del manejo de los hilos ya que este es más moderno y fácil de usar ya que evita el tener que crear e iniciar los hilos uniéndolos para que actúen simultáneamente de manera manual. Por lo tanto evita bastantes errores y es ideal para el procesamiento de información por lotes de información

- **Concurrent.future** Lo que realiza es establecer una rutina de funcionamiento ejecutando las funciones de los hilos de manera asíncrona usando los Threads y los procesos del sistema, por ello será como el gestor del funcionamiento del código. Con el **executor** lo que ahora es asignar el número de hilos para trabajar y ejecutarse en un recurso al mismo tiempo, para luego **Future** almacenar o reclutar la información obtenida en la ejecución del hilo y **executor.map** será quien bloquee y retorne las respuestas hasta que termine el trabajo.

1.2 Código implementado con Streamlit

```
1 import streamlit as st
2 from threading import Lock
3 from concurrent.futures import ThreadPoolExecutor
4 import string
5 import time
6
7 # Diccionario de palabras
8 positive_words = {"bueno", "excelente", "maravilloso", "genial", "fantástico", "encantó", "buenísimo", "lindo", "bono"}
9 negative_words = {"malo", "terrible", "horrible", "pésimo", "desastre", "malísimo", "horrendo", "feo"}
10 # Lock para evitar condiciones de carrera
11 lock = Lock()
12
13 # Funciones de analisis
14 def clean_text(text):
15     """Limpia puntuación y pone todo en minúscula."""
16     return text.lower().translate(str.maketrans("", "", string.punctuation))
17
18 def analyze_comment(comment):
19     """Analiza un solo comentario y retorna el sentimiento."""
20     words = clean_text(comment).split()
21     sentiment = "neutral"
22     if any(word in positive_words for word in words):
23         sentiment = "positive"
24     elif any(word in negative_words for word in words):
25         sentiment = "negative"
26     return (comment, sentiment)
27
28 def process_comments_parallel(comments, chunk_size, num_threads):
29     """Aca procesa comentarios en paralelo en chunks usando ThreadPoolExecutor y Lock."""
30     results = []
31
32     # Buscar dividir en chunks
33     chunks = [comments[i:i+chunk_size] for i in range(0, len(comments), chunk_size)]
34
35     def process_chunk(chunk):
36         local_results = []
37         for comment in chunk:
38             local_results.append(analyze_comment(comment))
```

Figure 3: Porción de código streamlit

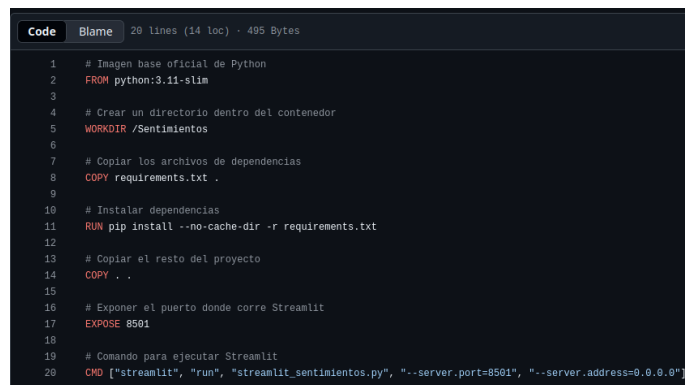
```
39 # Guardar los resultados globales con Lock
40 with lock:
41     results.extend(local_results)
42
43 # Ejecutar cada chunk en paralelo con hilos
44 with ThreadPoolExecutor(max_workers=num_threads) as executor:
45     executor.map(process_chunk, chunks)
46
47 return results
48
49 # Streamlit UI
50
51 st.title("Análisis de Sentimientos en Paralelo (Threading)")
52
53 st.write("Ingresa varios comentarios separados por líneas. La app procesará los comentarios en paralelo.")
54
55 text_input = st.text_area("Comentarios", height=80)
56 num_threads = st.slider("Número de hilos:", 1, 10, 3)
57 chunk_size = st.slider("Tamaño de cada chunk:", 1, 5, 2)
58
59 if st.button("Procesar"):
60     if not text_input.strip():
61         st.error("Por favor escribe al menos un comentario.")
62     else:
63         comments = [c.strip() for c in text_input.split("\n") if c.strip()]
64         st.info("Procesando los comentarios en paralelo...")
65
66         start_time = time.time()
67         results = process_comments_parallel(comments, chunk_size, num_threads)
68         end_time = time.time()
69
70         st.success("Procesamiento completado! :)")
71         st.write(f"Tiempo total: {end_time - start_time:.4f} segundos")
72
73 # Mostrar resultados
74 st.subheader("Resultados del Análisis")
75 st.table([{"Comentario": [r[0] for r in results],
76          "Sentimiento": [r[1] for r in results]})
77
78 # Conteo de sentimientos
79 counts = {"positive":0, "negative":0, "neutral":0}
```

Figure 4: Porción de código streamlit

- Para la implementación de los códigos tenga en cuenta los siguientes conceptos:
 - **Hilos:** Un hilo es quien puede ejecutar una pequeña rutina o trabajo en paralelo a otras rutinas, por ello se usa para tareas o trabajos concurrentes y pequeños permitiendo que hagan varios trabajos al tiempo.
 - **Condición de carrera:** Una condición de carrera se presenta cuando 2 o mas hilos intentan acceder a un recurso a la vez, por lo tanto de ser así se puede presentar una pérdida de información o una alteración de esta misma.
 - **Lock:** Es quien protegerá y evitará esa condición de carrera, siendo este una protección para cuando otro hilo quiera acceder a un recurso que ya está trabajando en conjunto con otro hilo, por eso se le asigna al hilo un `lock.acquire()` donde tomará el candado y su turno para acceder al recurso y de ser usado por otro hilo este esperará, `lock.release()` lo que realizará es liberar el candado.

1.3 Dockerización.

Procedemos a crear el archivo Dockerfile para la respectiva Dockerización.



```
Code Blame 20 lines (14 loc) · 495 Bytes
1 # Imagen base oficial de Python
2 FROM python:3.11-slim
3
4 # Crear un directorio dentro del contenedor
5 WORKDIR /Sentimientos
6
7 # Copiar los archivos de dependencias
8 COPY requirements.txt .
9
10 # Instalar dependencias
11 RUN pip install --no-cache-dir -r requirements.txt
12
13 # Copiar el resto del proyecto
14 COPY . .
15
16 # Exponer el puerto donde corre Streamlit
17 EXPOSE 8501
18
19 # Comando para ejecutar Streamlit
20 CMD ["streamlit", "run", "streamlit_sentimientos.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

Figure 5: Dockerfile.

respectivamente se procede a contruir la imagen, se debe aclarar que el contenedor se ejecutara en el puerto 8501 a través de streamlit, la construcción de la imagen seria la siguiente:

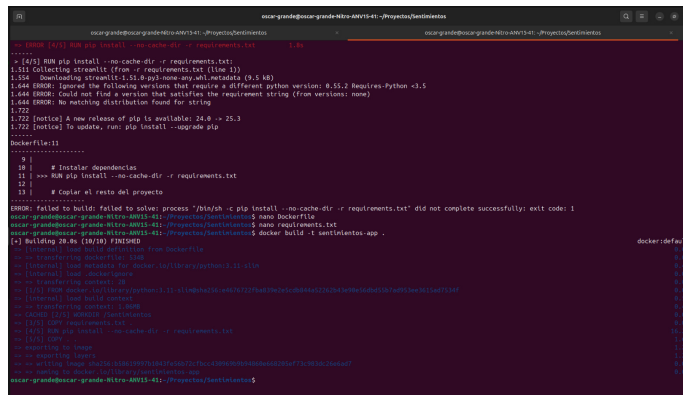


Figure 6: Dockerbuild

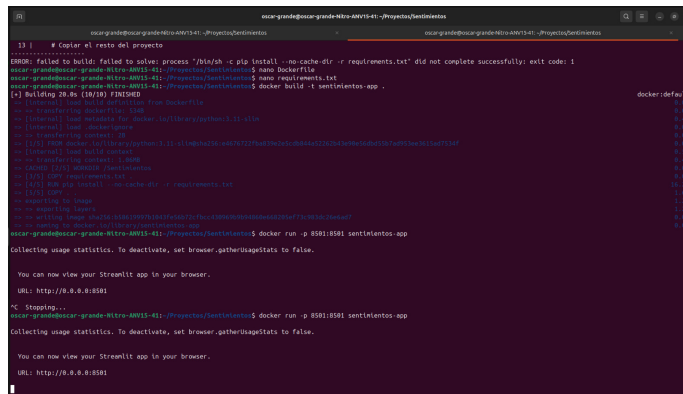


Figure 7: Ejecución de Docker.

Una vez se ejecute el contenedor este se podrá visualizar mediante el puerto 8501, lo bueno es que la terminal le da directamente el URL para poder ingresar a la visualización del dashboard, mientras el contenedor no este en ejecución no podremos visualizar el dashboard. Veriamos lo siguiente.

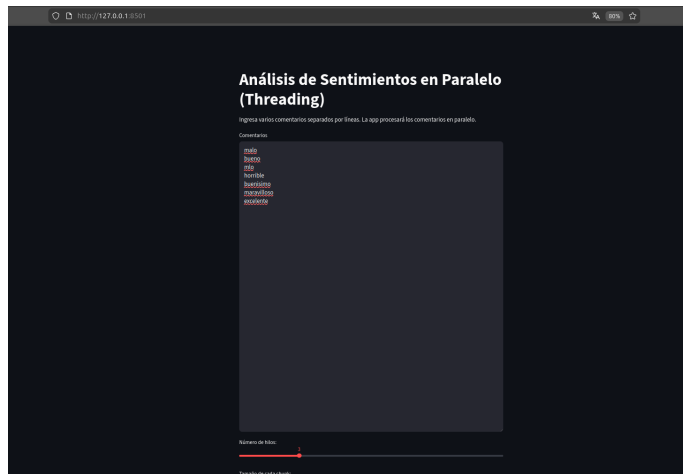


Figure 8: Visualización Dashboard

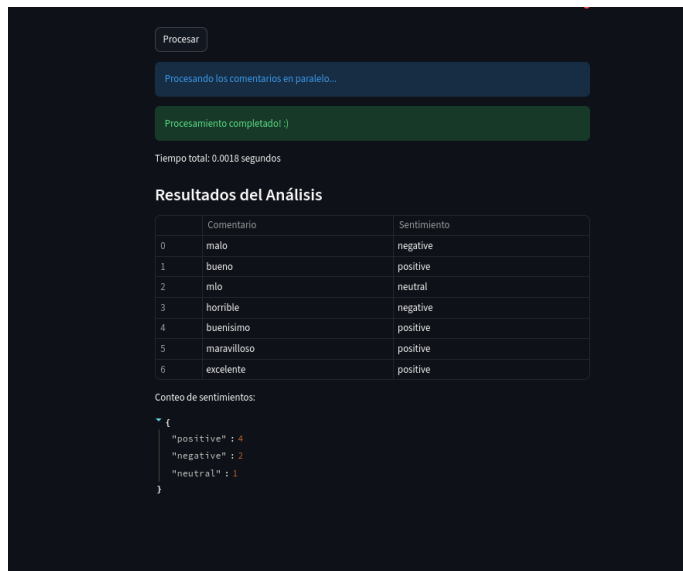


Figure 9: Visualización Dashboard.

Para aclarar la presencia de Chunks, es porque cuando se tiene una gran cantidad de comentarios en la cajita de comentarios, lo que se hará es la selección de hilos y chunks. Ya que si se le asigna un hilo a cada comentario lo que hará es hacer uso ineficiente por hilo, y si le asigna todos los comentarios a un solo hilo lo que hará es no aprovechar la "paralización" o el trabajo simultaneo de mas hilos. Por ello al emplear chunks y de-

terminar un valor es decirle al hilo que procese la información por cada chunk con una cantidad de un numero de cada comentarios, esto quiere decir que este funcionaria como una caja con una cantidad de comentarios y repartidos en los hilos que usted quiera disponer donde se guardar los comentarios y luego se almacenan de forma segura con el LOCK.

2 Ejercicio 2 (Juego 2D Mario Bross)

Partiremos de un código base del video juego, de allí partimos para mejorar las siguientes mecánicas del juego de este código base, implementando hilos, secciones criticas, semáforos y Mutex:

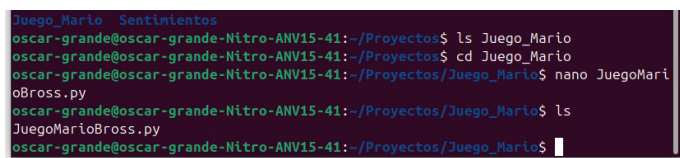
- Implementación de hilos y mutex con el fin de evitar o reparar bugs del jugador. Con saltos un poco mas controlados, colisiones mas precisas mejorando y realizando un trabajo mas eficiente, añadiendo mejoras para el comportamiento de los recursos entre los hilos. Mejorando a su vez las capaz de hilos fisicas, hilos de renderizado, hilos para los enemigos e hilo de spawnneo.
- Mejorar las mecánicas del juego, implementando los bloques destructibles, poderes (como el champiñon o la estrella), implementación y mejora de los enemigos con un scroll horizontal.

2.1 Explicación del Código.

El juego incluye una tarea paralela donde se ejecuta un análisis en segundo plano usando hilos. Esta tarea se sincroniza mediante un semáforo para proteger una variable compartida. Se incluye para demostrar el uso de hilos, mutex y secciones críticas sin afectar el ciclo principal del juego. Pued ever el codigo completo en el Gihub JuegoMario/JuegoMarioBross.py

2.2 Proceso.

- ingresamos al directorio de Proyectos en el cual vamos a crear la carpeta especifica para el juego, con el fin de que no se vaya a mezclar los contenidos de otro proyectos, allí crearemos el archivo JuegoMarioBross.py donde ira el código incluido del juego mejorado.



```
Juego_Mario Sentimientos
oscar-grande@oscar-grande-Nitro-ANV15-41:~/Proyectos$ ls Juego_Mario
oscar-grande@oscar-grande-Nitro-ANV15-41:~/Proyectos$ cd Juego_Mario
oscar-grande@oscar-grande-Nitro-ANV15-41:~/Proyectos/Juego_Mario$ nano JuegoMarioBross.py
oscar-grande@oscar-grande-Nitro-ANV15-41:~/Proyectos/Juego_Mario$ ls
JuegoMarioBross.py
oscar-grande@oscar-grande-Nitro-ANV15-41:~/Proyectos/Juego_Mario$
```

Figure 10: Proceso desde la terminal.

ahora se procede a crear y activar el entorno virtual dentro del proyecto, para luego realizar la instalación de pygame el cual es necesario ya que este se emplea en el código del juego por lo tanto al no tenerlo este no funcionara. Una vez instalado creamos los archivos requeridos Dockerfile y requerimientos.txt los cuales se verán su contenido mas adelante.

```
oscar-grande@oscar-grande-Nitro-ANV15-41:~/Proyectos/Juego_Mario$ ls
JuegoMarioBross.py
oscar-grande@oscar-grande-Nitro-ANV15-41:~/Proyectos/Juego_Mario$ python3 -m venv venv
/usr/bin/python3: No module named venv
oscar-grande@oscar-grande-Nitro-ANV15-41:~/Proyectos/Juego_Mario$ python3 -m venv venv
oscar-grande@oscar-grande-Nitro-ANV15-41:~/Proyectos/Juego_Mario$ source venv/bin/activate
(venv) oscar-grande@oscar-grande-Nitro-ANV15-41:~/Proyectos/Juego_Mario$ pip install pygame
Collecting pygame
  Downloading pygame-2.6.1-cp313-cp313-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (12 kB)
    Downloading pygame-2.6.1-cp313-cp313-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (14.0 MB)
      14.0/14.0 MB 40.8 MB/s eta 0:00:00
Installing collected packages: pygame
Successfully installed pygame-2.6.1
(venv) oscar-grande@oscar-grande-Nitro-ANV15-41:~/Proyectos/Juego_Mario$ nano Dockerfile
(venv) oscar-grande@oscar-grande-Nitro-ANV15-41:~/Proyectos/Juego_Mario$ nano requerimientos.txt
(venv) oscar-grande@oscar-grande-Nitro-ANV15-41:~/Proyectos/Juego_Mario$ ls
Dockerfile  JuegoMarioBross.py  requerimientos.txt  venv
(venv) oscar-grande@oscar-grande-Nitro-ANV15-41:~/Proyectos/Juego_Mario$
```

Figure 11: Proceso entorno virtual.

creamos el archivo Docker-compose.yml con el fin de crear la imagen automáticamente para poder ejecutar el contenedor y montar el código de la carpeta local dentro del contenedor para luego fácilmente reiniciar, detener, eliminar. Docker compose se emplea mas para cuando se tiene múltiples proyectos o detrás de la app se necesita manejar una base de datos o un servidor web, etc. En nuestro caso se empleo mas con el fin educativo y pedagógico, ya que al ser solo una app no es tan necesario este mismo, algunas de sus ventajas son :

- Puede editar el codigo local y se vera reflejado en el contenedor.
- Se puede añadir mas contenedores a esta imagen fácilmente.
- Comando de construcción mas corto.
- Manejo mas profesional para proyectos.

```
(venv) oscar-grande@oscar-grande-Nitro-ANV15-41:~/Proyectos/Juego_Mario$ nano Docker-compose.yml
(venv) oscar-grande@oscar-grande-Nitro-ANV15-41:~/Proyectos/Juego_Mario$
```

Figure 12: Proceso Docker compose

Ahora se procederá a la construcción de la imagen.

[illegible]

Figure 13: Construcción imagen Docker-compose.

Al tratarse de un video juego un poco mas implementado, este proceso puede tardar bastante, por ello se debe usar el paquete de python normal (slim no).

[illegible]

Figure 14: Construcción completada

Una vez completada el proceso tendremos como resultado el juego en ejecución y la visualización de su funcionamiento donde podremos ver:

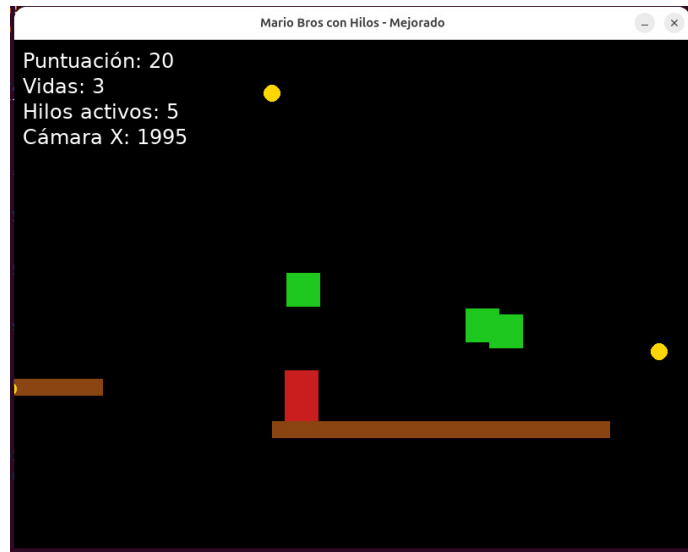


Figure 15: Juego en ejecución.

Cámara x quiere representar el scroll de movimiento por el mapa del juego, también podemos evidenciar en la siguiente imagen la presencia de un cuadrado rojo, el cual este es uno de los power-ups donde no nos afectara el daño de los enemigos una vez lo adquiramos, este es generado una vez golpeamos un bloque desde el inferior de este.

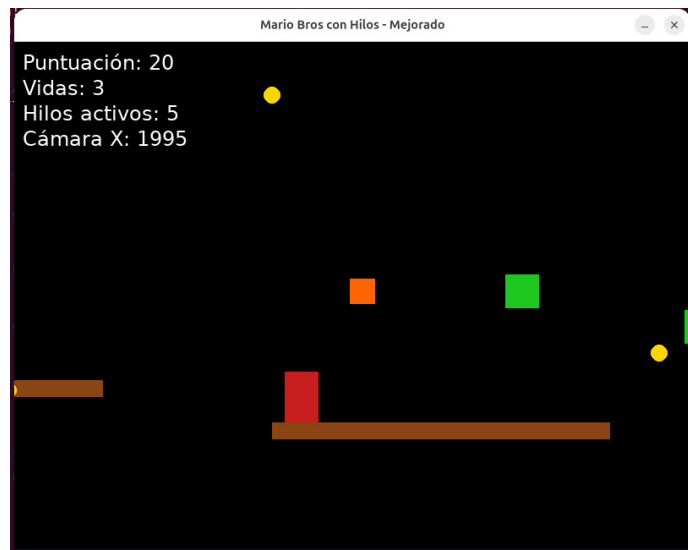


Figure 16: Power-up

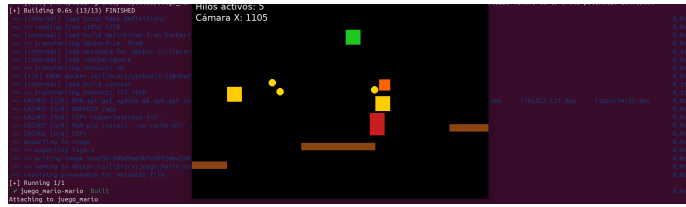


Figure 17: Poder generado por golpe

como evidencia se muestra el funcionamiento de estos juntos y el desplazamiento por el mapa.

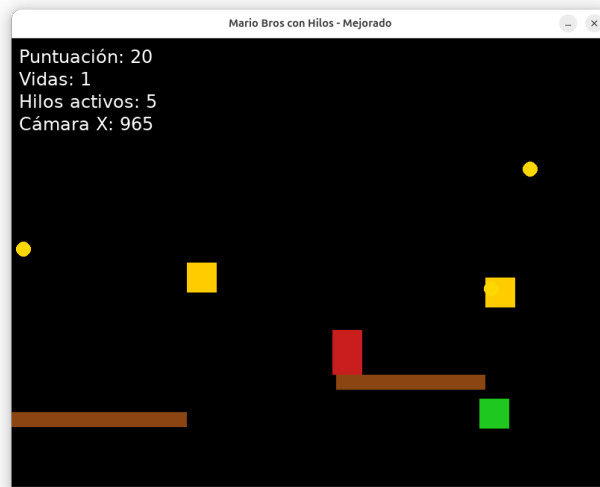


Figure 18: Desplazamiento por el juego

2.3 Dockerfile.

[illegible]

Figure 19: Dockerfile explicado.

2.4 Requerimientos.txt

[illegible]

Figure 20: Requerimientos.txt con dependencias.

2.5 Docker-compose

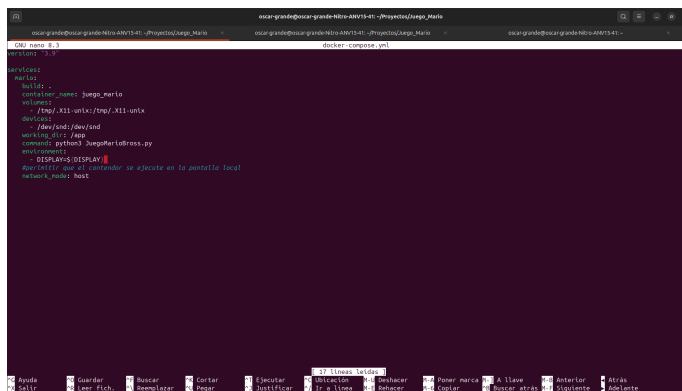


Figure 21: Docker-compose

3 Ejercicio 3.

Partiremos de un código base para desarrollar el algoritmo de detección de gestos de mano y este sea desplegado en streamlit, por ello es importante reconocer los términos y librerías que se van implementar (hilos, mutex, semáforos, secciones críticas). Pero en este ejercicio se añadirá mediapipe.

- **Mediapipe que es:** Es una colección de soluciones y modelos de visión a través de computador y ML en tiempo real desarrollada por google, donde en python se usa como biblioteca que facilitara la detección y el seguimiento de las diferentes partes del cuerpo humano o objetos en 3D, todo esto sucede en tiempo real sobre el video o cámaras sin la necesidad de entrenar modelos.

Esto brindara al usuario una facilidad de su implementación con una API de alto nivel para producción, optimizado en tiempo real (dependiendo de la plataforma, CPU y GPU), con modelos ya entrenados que funcionan en tareas comunes de detección con una licencia permisiva de apache 2.0.

- Puede realizar la detección de manos, face mash (cientos de puntos en la cara), detección de posiciones y ángulos del cuerpo, o la combinación de estas mismas.

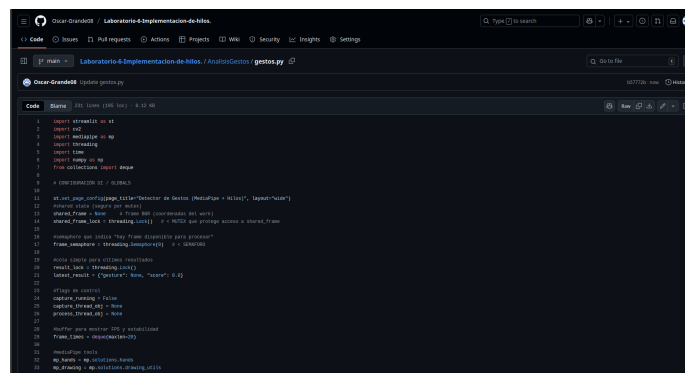
Una vez tengamos el concepto de esta nueva librería a implementar podemos avanzar en el desarrollo, a continuación se explicara la implementación de cada requerimiento para este ejercicio.

3.1 Código .py

- **Hilos:** se usara multiples hilos con una funcion distinta para cada uno:

- Un hilo realizara la captura de video.
 - Otor hilo realizara el procesamiento y reconocimiento.
 - otro hilo actualizara el estado y la logica.
 - Y el otro hilo quien sera el principal sera quien este mostrando el video en Streamlit con los resultados.
- **Mutex:** Sera quien protegerá el acceso a un frame compartido.
 - **Semaforo:** Sera quien controle el acceso cuando el procesador pueda leer el frame.
 - **Sección critica:** es la zona en la que se encontrara el frame procesado y donde se escribe y lee.

Para la explicación completa del código puede verla en /AnálisisGestos/Gestos.py en GITHUB donde encontrara la información completa de este y su funcionamiento.



```

1 import streamlit as st
2 import cv2
3 import numpy as np
4 import threading
5 import time
6 import sys
7 from collections import deque
8
9 # Configuración de la interfaz
10 st.title("Laboratorio 4 Implementación de hilos")
11 st.subheader("Gestos.py")
12
13 # Variables globales
14 shared_frame = None
15 shared_lock = threading.Lock()
16
17 # Función para capturar el video
18 def capture_video():
19     cap = cv2.VideoCapture(0)
20     while True:
21         ret, frame = cap.read()
22         if not ret:
23             continue
24         with shared_lock:
25             shared_frame = frame
26         time.sleep(0.01)
27
28 # Función para procesar el video
29 def process_video():
30     while True:
31         with shared_lock:
32             frame = shared_frame
33             if frame is None:
34                 continue
35             # Procesamiento de la imagen
36             # ... (código de procesamiento) ...
37
38 # Función para actualizar el estado
39 def update_state():
40     while True:
41         # Actualización del estado
42         # ... (código de actualización) ...
43
44 # Ejecución de los hilos
45 if __name__ == "__main__":
46     capture_thread = threading.Thread(target=capture_video)
47     process_thread = threading.Thread(target=process_video)
48     update_thread = threading.Thread(target=update_state)
49     capture_thread.start()
50     process_thread.start()
51     update_thread.start()
52
53 # Interfaz de usuario
54 st.image(shared_frame, width=640, height=480)
55
56 # Botón para detener la captura
57 if st.button("Detener"):
58     capture_thread.join()
59     process_thread.join()
60     update_thread.join()
61 
```

Figure 22: Codigo.py

3.2 Dockerfile.

En el archivo Dockerfile es donde ira el "paquete" de librerias de lenguaje python como media pipe y streamlit.

```
FROM python:3.11

# Instalar dependencias del sistema para matplotlib y ipynb
RUN apt-get update && apt-get install -y \
    build-essential \
    cmake \
    libblas3-base \
    liblapack-dev \
    libomp-dev \
    libscotch-dev \
    libxerces-c-dev \
    libxml2-dev \
    vim \
    libssl-dev \
    rtmpio \
    && rm -rf /var/lib/apt/lists/*

# Estructura de trabajo
WORKDIR /app

# Copiar dependencias
COPY requirements.txt .

# Instalar dependencias de Python
RUN pip install --no-cache-dir -r requirements.txt

# Copiar aplicación
COPY . .

# Exponer Streamlit
EXPOSE 8080

# Ejecutar Streamlit
CMD ["streamlit", "run", "gestos.py", "--server.address=0.0.0.0"]
```

Figure 23: Dockerfile.

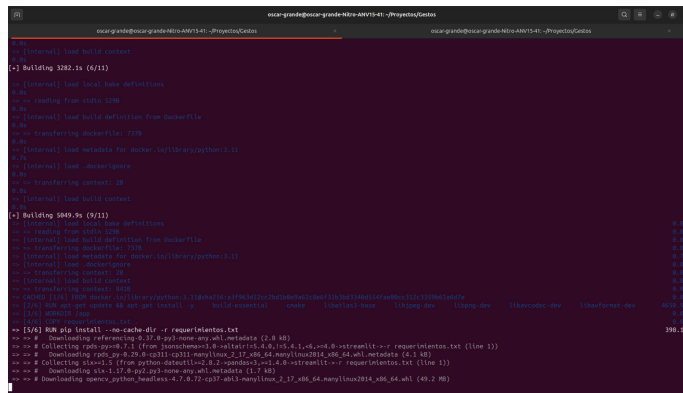
3.3 Docker compose.

El docker compose lo empleamos con el fin de poder subirlo fácilmente, además de que lo estamos subiendo a un dashboard con código python.

```
name: docker-compose.yml
services:
  gestos:
    build: .
    container_name: gestos_app
    volumes:
      - /dev/vid0:/dev/vid0
    ports:
      - 8080:8080
    environment:
      - STREAMLIT_SERVER_HEADLESS=true
    network_mode: host
```

Figure 24: Docker compose.

Una vez construido el archivo docker-compose y ejecuta para crear la imagen este puede tardar bastante, ya que interactúa con librerías pesadas por lo tanto la conexión debe ser fuerte y estable para el rápido proceso de esta.



3.4 Requerimientos.txt

En el archivo requerimientos.txt vamos añadir las librerías de las cuales van a depender el funcionamiento del código, estas son las que se emplean en el código Gestos.py

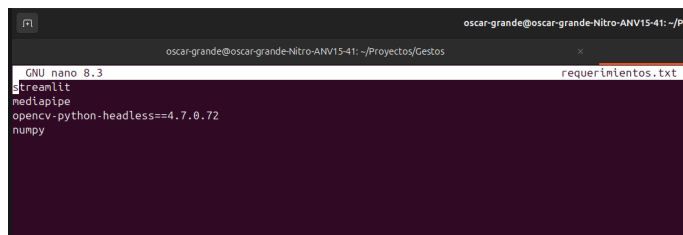


Figure 26: Requerimeintos.txt

3.5 Resultados.

Realizado todos los archivos necesarios podemos empezar con docker compose up –build para empezar la construcción de la imagen, esta puede tardar una vez realizada esta se ejecutara al instante donde podrá ver el resultado de lo obtenido.

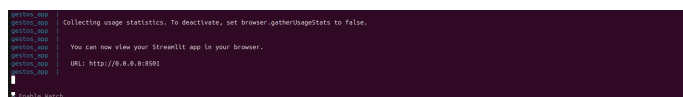


Figure 27: Como visualizar?

Una vez ejecutado y construida la imagen, en la terminal podrá ver el URL donde podrá ingresar o copiar y pegar en el navegador para abrir la ventana

de visualización del dashboard y así poder ver el programa en funcionamiento, ingrese.

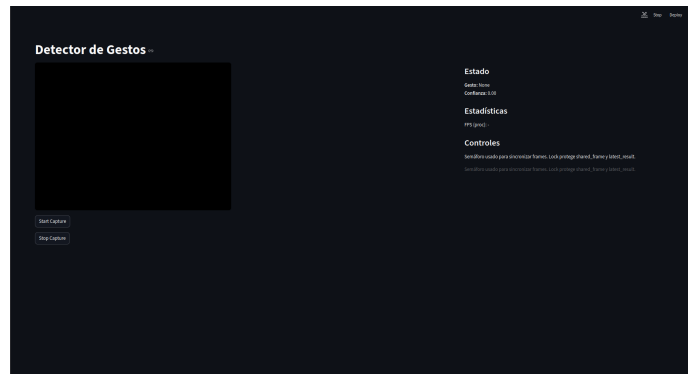


Figure 28: Visualización streamlit

Una vez ingrese al link vera el dashboard en funcionamiento, allí podrá interactuar con las opciones de iniciar captura y parar la captura. En el costado derecho vera el estado de funcionamiento con la información de si detecta un gesto o no, que tipo de gesto detecta y cual es la confianza de detección de este.

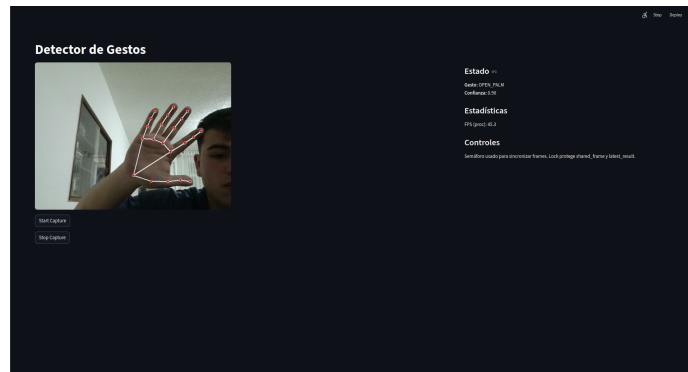


Figure 29: Funcionamiento

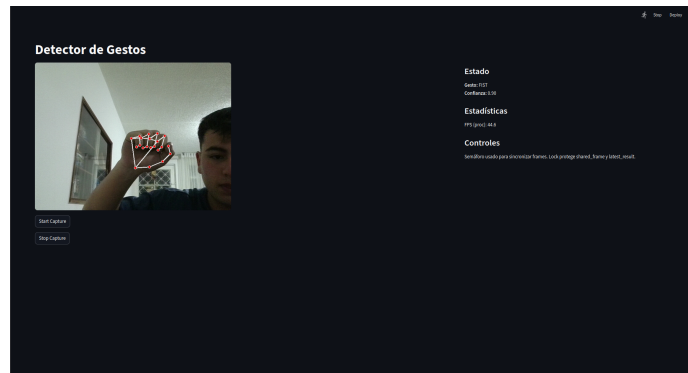


Figure 30: Funcionamiento.